OS ASSIGNMENT ORANGE -2

NAME: SUNIL S

SRN: PES1UG24CS634

SECTION: 4K

PHASE 1:

Screenshot 1A:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ make clean
make test_objects
./test_objects
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_obj
ects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:209:5: warning: ignoring return value of 'fread' declared with attribut
e 'warn_unused_result' [-Wunused-result]
  209 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$
```

Screenshot 1B:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$
```

PHASE 2:Screenshot 2A AND Screenshot 2B:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ make clean
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:209:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  209 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'write_tree_level':
tree.c:185:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 511 [-Wstringop-truncation]
  185 |             strncpy(te->name, rel, sizeof(te->name) - 1);
      |             ^
tree.c:175:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  175 |             strncpy(te->name, dir_name, sizeof(te->name) - 1);
      |             ^
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function 'index_add':
index.c:194:9: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
  194 |     if (object_write(OBJ_BLOB, data, size, &id) != 0) {
      |         ^~~~~~~~~~~~
index.c: In function 'index_load':
index.c:144:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  144 |     fread(&idx->count, sizeof(int), 1, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
index.c:145:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  145 |     fread(idx->entries, sizeof(IndexEntry), idx->count, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
index.c: In function 'index_add':
index.c:190:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  190 |     fread(data, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ find .pes/objects -type f
.pes/objects/ae/5a6723c82fc1b5042b589cba28dd652566ba72e498a86c7198698a654ec8e1
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/0fa523ffdd0baba81a81907ac82b8dbd0732cae0de8ac51db961f38ba3038a
.pes/objects/57/aeeb84256e3515a5320855f6aff651f6278b554107cd8e268584e4cdea265d
.pes/objects/71/c5382acd7bd1fb1a618253220941fa6c56c956e89405dd0b5376ee235983bb
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/e0/985fafd792331fa49279270483733544c89d6fe98e7c7c668df6087b4b3e27
.pes/objects/bd/7232fbc1232c3d938c2cda7bf723d8f15b064b03fa6b84b9e6eba39ad7b177
.pes/objects/31/2204d2670b37b626a054a6c189f6b97324b724ed3f5ec6c1bb797a42b88eb4
.pes/objects/b7/ef3c42aacbd53f8f61125b17c621cebaa7baee3b0928fc4a932df9799459ac
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
VBox_GAs_7.1.8
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ ^C
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ xxd .pes/objects/ae/5a6723c82fc1b5042b589cba2
8dd652566ba72e498a86c7198698a654ec8e1 | head -20
00000000: 636f 6d6d 6974 2032 3436 0074 7265 6520  commit 246.tree
00000010: 6237 6566 3363 3432 6161 6362 6435 3366  b7ef3c42aacbd53f
00000020: 3866 3631 3132 3562 3137 6336 3231 6365  8f61125b17c621ce
00000030: 6261 6137 6261 6565 3362 3039 3238 6663  baa7baee3b0928fc
00000040: 3461 3933 3264 6639 3739 3934 3539 6163  4a932df9799459ac
00000050: 0a70 6172 656e 7420 3537 6165 6562 3834  .parent 57aeeb84
00000060: 3235 3665 3335 3135 6135 3332 3038 3535  256e3515a5320855
00000070: 6636 6166 6636 3531 6636 3237 3862 3535  f6aff651f6278b55
00000080: 3431 3037 6364 3865 3236 3835 3834 6534  4107cd8e268584e4
00000090: 6364 6561 3236 3564 0a61 7574 686f 7220  cdea265d.author
000000a0: 5045 5320 5573 6572 203c 7065 7340 6c6f  PES User <pes@lo
000000b0: 6361 6c68 6f73 743e 2031 3737 3633 3232  calhost> 1776322
000000c0: 3438 350a 636f 6d6d 6974 7465 7220 5045  485.committer PE
000000d0: 5320 5573 6572 203c 7065 7340 6c6f 6361  S User <pes@loca
000000e0: 6c68 6f73 743e 2031 3737 3633 3232 3438  lhost> 177632248
000000f0: 350a 0a49 6e69 7469 616c 2063 6f6d 6d69  5..Initial commi
00000100: 74                                       t
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ =
```

PHASE 3:

Screenshot 3A:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ ./pes init
./pes add a.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     a.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  index.c
  untracked:  object.c
  untracked:  pes.h
  untracked:  .gitignore
  untracked:  index.h
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  commit.h
  untracked:  commit.c
  untracked:  README.md
  untracked:  file1.txt
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  test_sequence.sh
```

Screenshot 3B:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321596 6 a.
txt
```

PHASE 4:

Screenshot 4A:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ ./pes log
commit 6f0fa523ffdd0baba81a81907ac82b8dbd0732cae0de8ac51db961f38ba3038a
Author: PES User <pes@localhost>
Date:   1776322503

    Add farewell

commit e0985fafd792331fa49279270483733544c89d6fe98e7c7c668df6087b4b3e27
Author: PES User <pes@localhost>
Date:   1776322494

    Add worldell

commit ae5a6723c82fc1b5042b589cba28dd652566ba72e498a86c7198698a654ec8e1
Author: PES User <pes@localhost>
Date:   1776322485

    Initial commit

commit 57aeeb84256e3515a5320855f6aff651f6278b554107cd8e268584e4cdea265d
Author: PES User <pes@localhost>
Date:   1776321596

    first commitirst commit>Q
```

Screenshot 4B:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/31/2204d2670b37b626a054a6c189f6b97324b724ed3f5ec6c1bb797a42b88eb4
.pes/objects/57/aeeb84256e3515a5320855f6aff651f6278b554107cd8e268584e4cdea265d
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/0fa523ffdd0baba81a81907ac82b8dbd0732cae0de8ac51db961f38ba3038a
.pes/objects/71/c5382acd7bd1fb1a618253220941fa6c56c956e89405dd0b5376ee235983bb
.pes/objects/ae/5a6723c82fc1b5042b589cba28dd652566ba72e498a86c7198698a654ec8e1
.pes/objects/b7/ef3c42aacbd53f8f61125b17c621cebaa7baee3b0928fc4a932df9799459ac
.pes/objects/bd/7232fbc1232c3d938c2cda7bf723d8f15b064b03fa6b84b9e6eba39ad7b177
.pes/objects/e0/985fafd792331fa49279270483733544c89d6fe98e7c7c668df6087b4b3e27
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$
```

Screenshot 4C:

```
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ cat .pes/refs/heads/main
6f0fa523ffdd0baba81a81907ac82b8dbd0732cae0de8ac51db961f38ba3038a
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
deepaK@virtualVM:~/PES1UG24CS634-pes-vcs$
```

Phase 5: Branching and Checkout

Q5.1: How to implement pes checkout <branch>

Answer:

A branch is stored as a file in .pes/refs/heads/ containing a commit hash.

To implement pes checkout <branch>:

1. Read the commit hash from .pes/refs/heads/<branch>
2. Update .pes/HEAD to point to this branch
3. Load the commit object using the hash
4. Extract the tree hash from the commit
5. Read the tree and reconstruct the working directory:
    - Delete current files
    - Write files from tree objects

Complexity:

- Must update many files correctly
- Must avoid overwriting uncommitted changes
- Must rebuild directory structure accurately

Q5.2: Detect dirty working directory

Answer:

To detect a dirty working directory:

1. For each tracked file in the index:
    - Read file contents from working directory
    - Compute its hash
    - Compare with stored hash in index
2. If any file hash differs:
    - Working directory is dirty

Conflict detection:
If a file is modified AND different in the target branch, checkout must be refused to prevent data loss.

Q5.3: Detached HEAD

Answer:

Detached HEAD means .pes/HEAD contains a commit hash instead of a branch reference.

What happens:

- New commits can still be created
- But no branch points to them

- These commits become unreachable

Recovery:
User can create a branch pointing to that commit:

git checkout -b new-branch

Conclusion:
Detached HEAD commits exist temporarily but can be lost if not referenced

Phase 6: Garbage Collection

Q6.1: Find and delete unreachable objects

Answer:

Algorithm:

1. Start from all branch heads:

.pes/refs/heads/*

2. Traverse all reachable objects:
    - Commit → Tree → Blob
3. Store visited hashes in a set
4. Scan .pes/objects
    - Delete objects not in the reachable set

Data Structure:
Use a **hash set** for efficient lookup.

Estimation:
For 100,000 commits and 50 branches:

- All commits, trees, and blobs must be visited

- Total objects ≈ few hundred thousand

Q6.2: Why GC is dangerous during commit

Answer:

Garbage collection during commit can cause race conditions.

Scenario:

1. Commit process creates new objects
2. GC runs at the same time
3. GC does not see new objects yet
4. GC deletes them as unreachable

Result:
Commit refers to deleted objects → repository corruption

How Git avoids this:

- Uses locking to prevent GC during commit
- Uses temporary objects before finalizing commit
- Ensures atomic updates